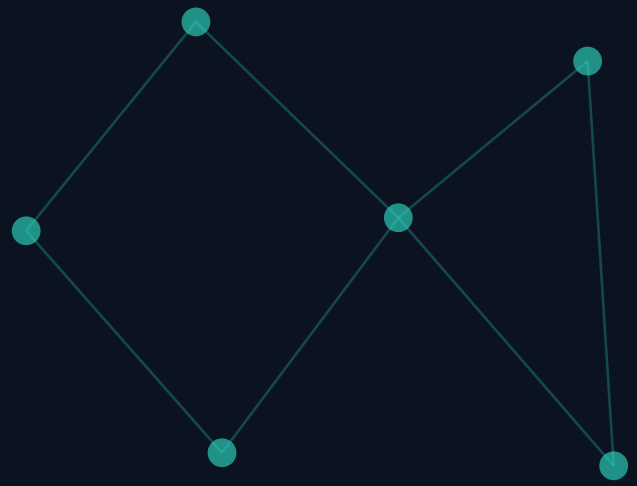
CLASE 7 · UNIDAD 3 · SEMANA 7



# Prompt Engineering

Dirigir el modelo desde el prompt, sin tocar los pesos: de few-shot y Chain-of-Thought a structured outputs y function calling.

***El prompt no reentrena:*** *condiciona una distribución ya entrenada.*

# Mismo modelo, salidas distintas

**El punto de partida.** Un mismo checkpoint —los mismos pesos, congelados— produce salidas radicalmente distintas según cómo le pedís las cosas. Cambiás la instrucción y cambia todo. Eso no es magia.

**Por qué pasa.** Un large language model (LLM) es una distribución condicional sobre tokens: dado un contexto, predice el próximo. Ya lo vimos en el Transformer. El prompt es el contexto — o sea, es la condición de esa distribución.

**La tesis de la que cuelga toda la clase.** Few-shot, Chain-of-Thought, structured outputs, function calling: todo es dirigir esa distribución sin tocar un solo peso. No reentrenás: condicionás.




$$P(\text{próximo token} \mid \text{contexto})$$

*el prompt ES el contexto = la condición de la distribución*

**La idea que hay que fijar primero.** El modelo no cambia; cambia lo que le entra. Prompt engineering es ingeniería sobre la condición de una distribución que ya está entrenada. Sin gradientes, sin datos nuevos: sólo el contexto.

# Cambiar lo que HACE, no lo que SABE

**Dos maneras de adaptar un modelo base.** En la Clase 5 partimos un modelo base por dos ejes: conocimiento (continued pretraining) y comportamiento (supervised fine-tuning / RLHF / DPO). Los dos tocan los pesos: reentrenan.

**El prompt es una tercera vía.** No toca los pesos. No agrega conocimiento nuevo ni cambia lo aprendido: cambia lo que el modelo hace con lo que ya sabe, en este contexto puntual. Es adaptación en inference-time.

**Cuándo cada uno (criterio de AI Engineering).** El prompt es más rápido, barato, legible, preserva el conocimiento general y sobrevive a las actualizaciones del modelo. El fine-tuning se justifica cuando el prompt no alcanza: comportamiento muy específico, formato rígido a gran volumen, latencia.




**La distinción operativa.** Fine-tuning cambia lo que el modelo sabe/es; el prompt cambia lo que hace ahora. Empezá siempre por el prompt: es la palanca barata y reversible. El fine-tuning es el recurso de después, no el primero.

# El mapa: tres lugares donde intervenir

**El eje único.** Todo lo de hoy es dirigir la distribución del próximo token sin tocar los pesos. El modelo ya está entrenado; nosotros sólo moldeamos, desde afuera, de dónde sale cada token.

**Tres lugares donde intervenir.** (1) El prompt —lo que le mostrás y le pedís—: las técnicas de prompting. (2) El decoder —cómo lo dejás generar—: structured outputs. (3) Por encima de ambos: automatizar el prompt y curar todo el contexto — el escalón meta.

**El primero, por dentro.** Prompting son las seis familias del campo, y se parten en dos operaciones: condicionar (qué le mostrás) y expandir (cuánto lo dejás pensar: razonar, buscar, deliberar, revisar).




**Un modelo congelado, tres lugares para dirigirlo.** El input, el decoder, y la capa de encima. El resto de la clase recorre los tres, en ese orden.

# Zero, one, few-shot: el espectro

**Arranquemos por lo mínimo.** Zero-shot: le pedís la tarea y listo, sin ejemplos. Funciona sorprendentemente bien para tareas comunes, porque el modelo ya vio millones de casos parecidos en pretraining.

**Sumá ejemplos y algo cambia.** One-shot: un ejemplo input→output antes del pedido. Few-shot: varios. Con dos o tres ejemplos, tareas que zero-shot resolvía mal empiezan a salir. El nombre viene de GPT-3 (Brown et al. 2022, Language Models are Few-Shot Learners).

**El error clásico, cortado de raíz.** “Shot” = cantidad de demostraciones en el contexto. NO es cantidad de intentos. Few-shot no es “probar varias veces”: es “mostrar varios ejemplos”.




*el pedido es idéntico; crecen los ejemplos →*

**La confusión a matar.** shot = demostración, no intento. Con esto ya tenemos el vocabulario que la próxima parte necesita: sobre el eje “cuántas demostraciones” vamos a montar un segundo eje.

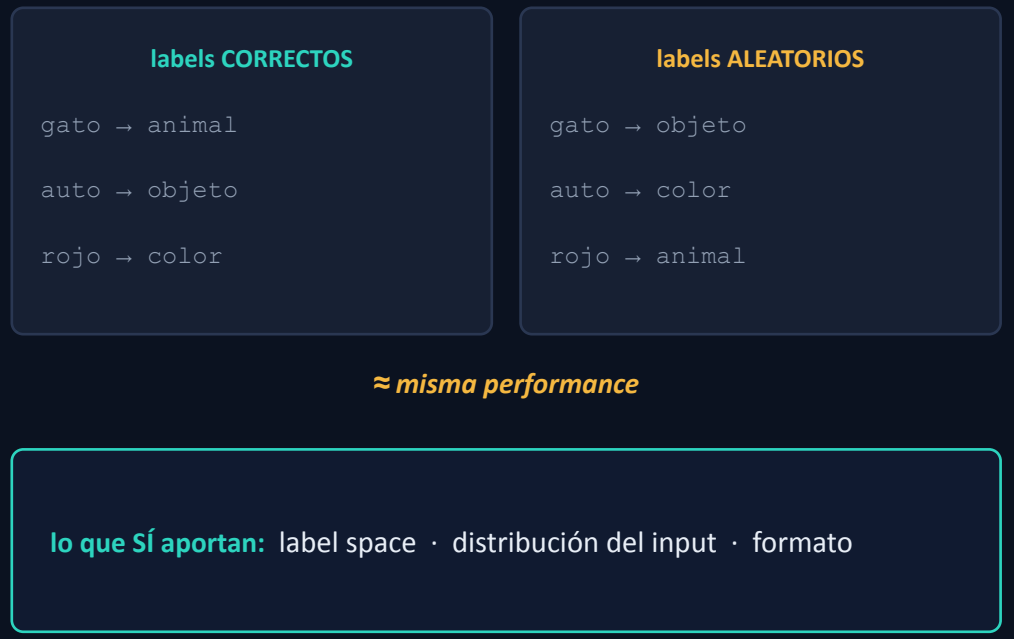
# Qué enseñan realmente los ejemplos

**La pregunta natural.** Si los ejemplos ayudan, es porque le enseñan las respuestas correctas… ¿no? Resulta que no, y el resultado es contraintuitivo.

**El experimento (Min et al. 2022).** Reemplazá los labels de las demostraciones por labels aleatorios. La performance apenas cae, consistente en 12 modelos incluido GPT-3. Los ejemplos no funcionan por el mapeo correcto input→respuesta.

**Qué fijan entonces.** Tres cosas: el espacio de labels (qué respuestas son posibles), la distribución del input (cómo se ven las entradas), y sobre todo el formato de la secuencia. Enseñan la forma de la tarea, no las respuestas.

**El matiz honesto.** Trabajos posteriores (Yoo et al. 2022) muestran que el efecto del ground-truth depende de la configuración. La lectura correcta: la forma domina, no "el contenido es irrelevante".




**La primera aparición del backbone.** Imitar demostraciones enseña la forma, no la capacidad — la misma distinción del tag <think> (Clase 6): el cold-start SFT enseñaba "modales", no a pensar. Los few-shot fijan el molde de la tarea; la capacidad ya estaba en los pesos.

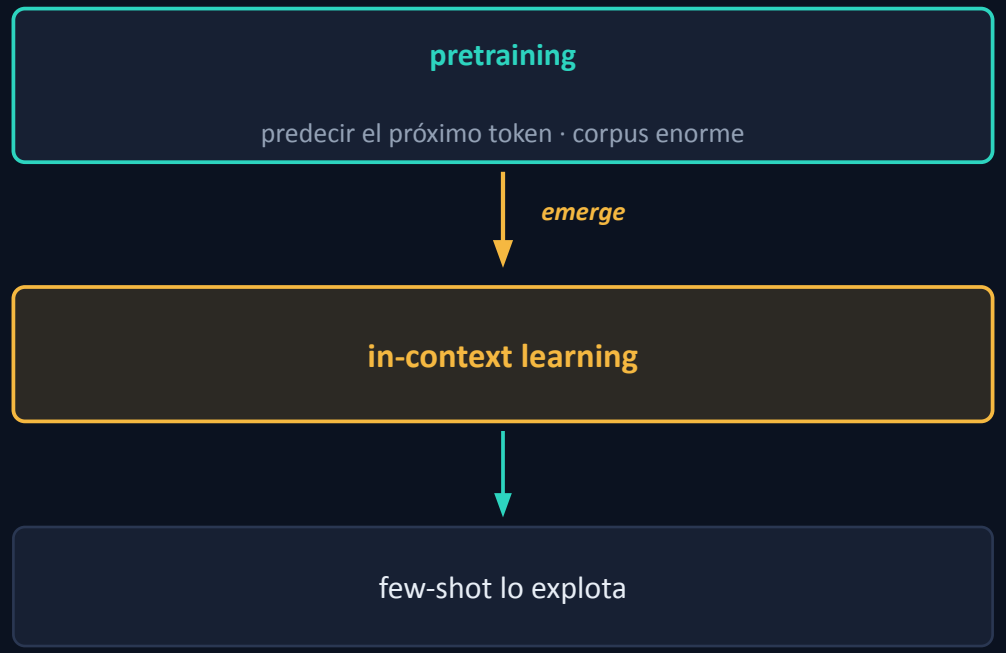
# Por qué funciona: emerge del pretraining

**El nombre del fenómeno.** Aprender una tarea a partir de ejemplos en el contexto, sin actualizar un solo peso, se llama in-context learning. Es lo que hace que los few-shot funcionen.

**El alcance (para que no se lea como "más sobre few-shot").** In-context learning es el paraguas, no un sinónimo de few-shot: todo el espectro zero→few vive acá adentro, y también seguir una instrucción, copiar un formato mostrado una vez o aprovechar un dato pegado en el prompt son el mismo fenómeno —el modelo dejándose guiar por el contexto—. Few-shot es la muestra más nítida, no la definición.

**No se entrena explícitamente: emerge.** Nadie entrenó al modelo "para aprender de ejemplos". El in-context learning emerge de la única tarea de pretraining —predecir el próximo token sobre un corpus gigante (Clase 5)—. Predecir texto arbitrario obliga a inferir, sobre la marcha, qué tarea está en juego.

**Sigue sin entenderse del todo.** Es área de investigación abierta. Una pista mecanicista: circuitos internos (induction heads) que copian y completan patrones del contexto. La capacidad está en los pesos; el prompt sólo la invoca.




**Por qué esto es más que "cómo prompteo".** El few-shot no "le enseña" la tarea en el sentido de entrenar: activa una capacidad que el pretraining ya dejó instalada. El mismo principio reaparece con razonamiento y con structured outputs: la forma se invoca desde el prompt; la capacidad viene de los pesos.

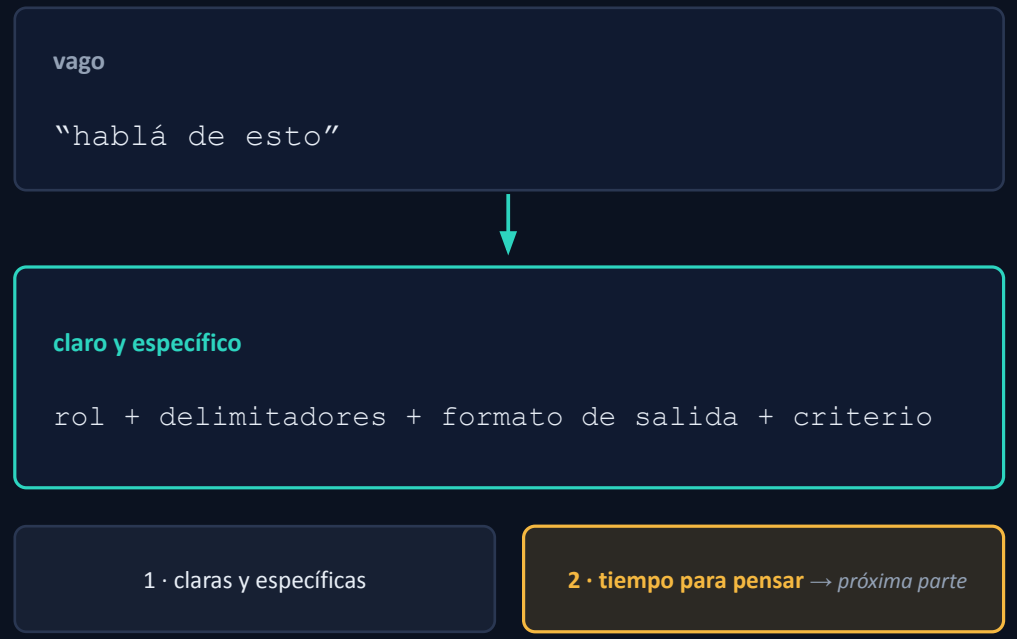
# Los dos principios del craft

**Bajemos a la práctica.** Toda la literatura de prompting colapsa, en su base, en dos principios (DeepLearning.AI, Fulford & Ng). Uno gobierna esta parte; el otro abre la que sigue.

**Principio 1 — instrucciones claras y específicas.** El modelo no lee entre líneas: respondé a lo que está escrito. Tácticas: delimitadores para separar input de instrucción, pedir el formato de salida explícito, dar un rol, estructurar en secciones.

**Principio 2 — darle tiempo para pensar.** No le pidas la respuesta de una: dale lugar para trabajar el problema antes de comprometerse. Es literalmente lo que la próxima parte convierte en técnica (Chain-of-Thought).

**Claridad, no brevedad.** El mito es “prompts cortos”. Lo que importa es claro y específico, no corto. Un prompt claro y largo le gana a uno vago y corto.




**Lo accionable de hoy.** Claras y específicas + tiempo para pensar. El resto de la clase son formas cada vez más sofisticadas de estos dos principios: el principio 2 se vuelve razonamiento; y “el formato de salida” se vuelve structured outputs.

# El campo se siente como una bolsa de trucos

**El problema, tal como se siente.** Prompt engineering parece una colección de tips inconexos: cada semana un nombre nuevo y pegadizo, sin criterio para saber si una técnica es fundamental o una variación menor de otra. Es fácil creer que a uno "le faltan técnicas".

**El objeto que lo ordena.** Un survey sistemático (Schulhoff et al. 2024, The Prompt Report): 32 investigadores relevaron más de 1.500 papers con metodología PRISMA —la de las revisiones sistemáticas en medicina— y destilaron 58 técnicas text-only, 40 para otras modalidades, y 33 términos de vocabulario estandarizado.

**Por qué la lista tiene peso.** No es el gusto de los autores sobre "las mejores": es el resultado de un procedimiento reproducible y auditable. Cualquiera que siga los mismos pasos llega a la misma lista. Y el vocabulario común ataca lo que venimos marcando: nombrar bien las cosas.




*trazable: cada etapa se puede reconstruir*

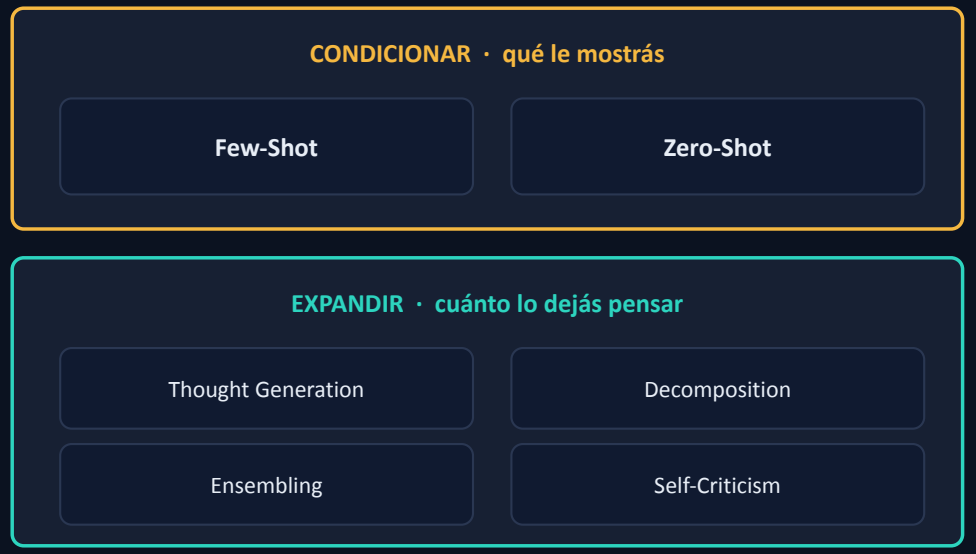
**El mapa es un hallazgo, no una opinión.** El catálogo sale de un procedimiento reproducible, no de una preferencia — por eso se le puede creer.

# Las 58 técnicas colapsan en seis familias

**El número que importa no es 58, es seis.** El survey agrupa las 58 técnicas de prompting en seis familias; y esas seis se parten limpio en las dos operaciones que organizan la clase: condicionar (Few-Shot, Zero-Shot) y expandir (las otras cuatro). Todo el prompting es una de las dos.

**Condicionar: qué le mostrás.** Few-Shot (ejemplos) y Zero-Shot (instrucción, rol, estilo) fijan la condición de la que el modelo genera. No agregan cómputo; cambian el punto de partida.

**Expandir: cuánto lo dejás pensar.** Thought Generation (razonar), Decomposition (partir), Ensembling (muestrear y agregar), Self-Criticism (criticar y revisar) gastan más tokens/cómputo. Es el próximo bloque.




**Todo el prompting es condicionar o expandir.** Dos operaciones sobre el input, seis familias — y ni una es restringir: eso es otro lugar del sistema (el decoder), la próxima parada grande.

# La grilla: demostraciones × razonamiento

**El segundo eje.** Hasta acá variamos cuántas demostraciones (zero/few-shot). Ahora agregamos un eje ortogonal: ¿la demostración —o la salida— incluye el paso a paso, o salta directo a la respuesta?

**Dos ejes, cuatro celdas.** Cruzando "cuántos ejemplos" con "¿muestra el razonamiento?" queda una grilla. Cada técnica famosa de esta parte es una celda de esa grilla, no un truco nuevo.

**Por qué importa el encuadre.** Sin este eje, Chain-of-Thought y "let's think step by step" se sienten como dos hacks aislados. Con el eje, son dos puntos de una grilla que ya tenés en la cabeza.




*↑ la fila de abajo: lo que sigue*

**El puente desde la parte anterior.** El eje de shots (cuántas demostraciones) y el de razonamiento (¿la salida muestra los pasos?) son independientes. Todo lo que viene se ubica en esta grilla: la técnica no es el punto — el mapa es el punto.

# Chain-of-Thought: mostrar el razonamiento

**La celda de abajo a la izquierda.** Chain-of-Thought (CoT) es few-shot donde las demostraciones incluyen los pasos intermedios, no sólo la respuesta final (Wei et al. 2022). Los mismos few-shot, pero mostrando cómo se llega.

**El salto.** En tareas de razonamiento —aritmética, lógica— la accuracy da un salto grande cuando los ejemplos exhiben el paso a paso. El modelo copia la forma de razonar y la aplica al problema nuevo.

**Aparece con la escala.** En modelos chicos CoT no ayuda; recién con escala suficiente la curva pasa de plana a creciente. Es una capacidad emergente: el prompting estándar sólo mostraba una cota inferior de lo que el modelo podía.




**El Aha! de esta parte.** CoT no le agrega conocimiento al modelo. Le da lugar para computar antes de comprometerse a una respuesta. Los tokens intermedios no son adorno: son el cálculo hecho visible. Por qué funciona, en la próxima lámina.

# Tokens como cómputo

**Por qué escribir más ayuda a pensar mejor.** Cada token que el modelo genera es un forward pass completo por la red. No hay un “scratchpad” interno oculto donde razonar en secreto: lo único que puede usar como memoria de trabajo es lo que ya escribió.

**La cadena ES el cómputo.** Al escribir los pasos, el modelo se da a sí mismo más forward passes y un lugar donde apoyar cálculos intermedios. Saltar a la respuesta obliga a resolver en la profundidad fija de una sola pasada —las capas son un número—; escribir los pasos encadena varias pasadas en una computación serial más larga.

**El eco de la Clase 6.** Con RLVR (Reinforcement Learning from Verifiable Rewards), premiar “acertar seguido” empujaba —sin querer— hacia respuestas más largas: pasos más chicos, más tokens, más confiable. La misma física: más tokens = más cómputo.




**La intuición mecánica.** No hay pensamiento sin tokens donde pensarlo. Por eso CoT ayuda, por eso el <think> de la Clase 6 ayuda, y —lo veremos— por eso restringir demasiado la salida puede ahogar el razonamiento: le sacás el lugar para computar.

# Releer la pregunta: cómputo en el input

**La otra punta de la secuencia.** La lámina anterior gastaba cómputo en la SALIDA. Se puede gastar también en el INPUT: Re-Reading (RE2, Xu et al. 2023) es, literalmente, repetir la pregunta en el prompt —Q: {pregunta} / "Leé la pregunta de nuevo: {pregunta}"— antes de pedir la respuesta.

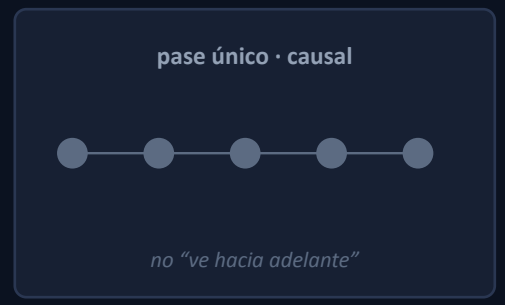
**Por qué funciona (y acá está el Aha!).** Un decoder-only es causal: en un solo pase, cada token de la pregunta sólo atiende a lo que vino ANTES, no a lo que sigue (Clase 2, la máscara causal). Al releer, en la segunda pasada cada token ya se apoya en la pregunta entera de la primera. Recuperás comprensión "bidireccional" —estilo BERT— en un modelo unidireccional, sólo con el prompt.

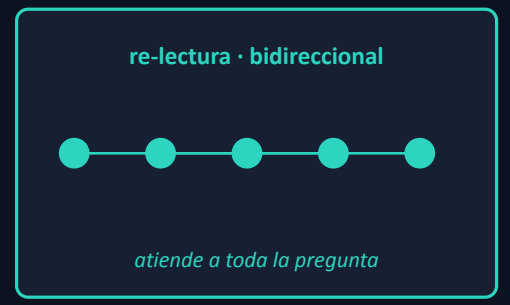
**Los detalles finos.** Releer la PREGUNTA ayuda; repetir el gatillo ("pensemos paso a paso") dos veces, no. Compone con CoT: es plug-and-play. Costo: alarga un poco el input.

```
Q: {pregunta}

Leé la pregunta de nuevo: {pregunta}

(gatillo CoT opcional)
```



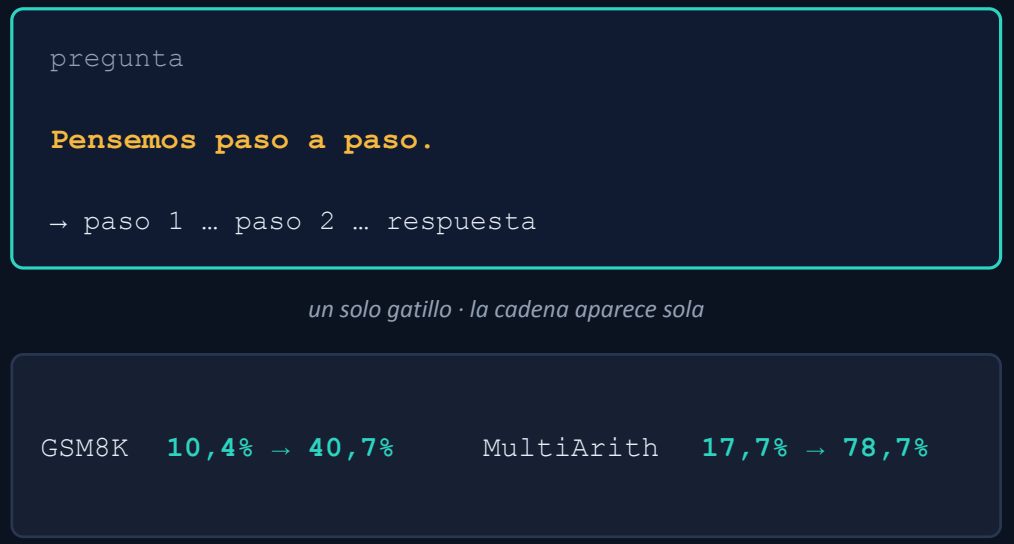

**Cómputo en el input, no en la salida.** CoT le da al modelo lugar para pensar la respuesta; RE2 le da lugar para entender la pregunta — burlando la máscara causal de la Clase 2. Dos palancas del mismo eje "más tokens = más cómputo", en las dos puntas de la secuencia.

# Zero-shot CoT: la frase gatillo

**La celda de abajo a la derecha.** ¿Y si querés el razonamiento sin armar ejemplos? Zero-shot CoT (Kojima et al. 2022): agregás una sola frase —"pensemos paso a paso"— y el modelo despliega la cadena solo, sin ninguna demostración.

**El salto, sin ejemplos.** Con ese único gatillo, GSM8K sube de 10,4% a 40,7% y MultiArith de 17,7% a 78,7% (InstructGPT). El mismo template sirve para aritmética, lógica y sentido común: es task-agnostic.

**Qué se impone y qué ya estaba.** La frase gatillo impone la forma (activá el modo "mostrar pasos"). La capacidad de razonar ya estaba en los pesos. Es, otra vez, forma-desde-el-prompt / capacidad-en-los-pesos — el patrón del tag <think>.




**La grilla, completa.** Ya tenemos las cuatro celdas: few-shot y zero-shot, cada uno con y sin razonamiento. La frase gatillo es andamiaje de formato; no le enseña a pensar, le dice cuándo mostrar que piensa.

# Self-Consistency: muestrear y votar

**Una cadena es una muestra, no una verdad.** El modelo es estocástico: samplea en cada token. Con temperatura > 0, la misma pregunta produce cadenas distintas, algunas correctas, otras no.

**La técnica (Wang et al. 2022).** Generá varias cadenas independientes para el mismo problema y quedate con la respuesta mayoritaria. En vez de confiar en un solo camino de razonamiento, marginalizás sobre muchos.

**Por qué mejora.** Hay muchos caminos que llegan a la respuesta correcta y pocos que coinciden en la misma respuesta incorrecta. Votar filtra el ruido de un camino aislado. Más muestras = más confiabilidad: escalar cómputo en inference-time.

**El matiz, para ubicarla en 2026.** Self-consistency es paralela: varias cadenas independientes que después votás. Sigue siendo útil y barata en modelos chicos como el nuestro (no necesita un modelo de scoring aparte), pero en los reasoning models el RL ya internalizó esta exploración — y, como veremos en dos láminas, el razonamiento secuencial interno escala mejor que votar en paralelo (lámina 12).



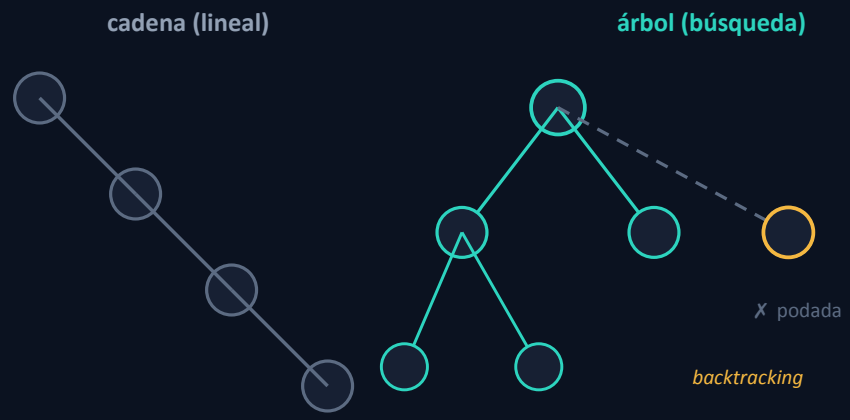

**La cadena es muestreable.** Tratar el razonamiento como una distribución de la que sacás varias muestras y agregás es un patrón que vuelve en agentes (deliberar, votar). Acá aparece la palanca general: gastar más cómputo en inferencia compra más confiabilidad.

# De la cadena al árbol

**CoT es lineal.** Una cadena avanza en una sola dirección: si un paso temprano sale mal, arrastra el error hasta el final. No hay vuelta atrás.

**Tree-of-Thought (Yao et al. 2023).** Generaliza la cadena a un árbol: en cada punto el modelo explora varias continuaciones, evalúa cuáles prometen, y puede retroceder (backtracking) por las ramas malas. Es búsqueda deliberada sobre estados de razonamiento, no un único trazo.

**Dónde termina esta línea.** Cuando la búsqueda incorpora acciones sobre el mundo —buscar, calcular, consultar— entramos en territorio de agentes (razonar-y-actuar, ReAct). Eso lo construimos en la unidad de agentes; acá dejamos marcado el puente.




*búsqueda sobre razonamiento → agentes*

**El escalón de complejidad.** CoT → self-consistency (varias cadenas) → Tree-of-Thought (un árbol con búsqueda). Cada paso gasta más cómputo para razonar mejor. El siguiente escalón —acciones y herramientas— es la puerta a los agentes.

# Cuando el modelo ya razona solo

**El cambio de época.** Todo lo anterior asume un modelo que no razona por default. Pero los reasoning models —o1, R1, o3, el modo think de Qwen3— ya fueron entrenados con RL para razonar solos (Clase 6, RLVR): la cadena ya está incorporada a los pesos.

**No sólo CoT: toda la familia.** El RL no internalizó únicamente Chain-of-Thought, sino también la exploración paralela (self-consistency) y la búsqueda (Tree-of-Thought): backtracking, autoevaluación y muestreo múltiple, todo adentro. Por eso, en un reasoning model, aplicarlas por fuera rinde poco o negativo (Prompting Science Report 2, 2025).

**Por qué lo secuencial le gana a lo paralelo.** El razonamiento interno escala mejor que votar en paralelo porque los pasos posteriores se apoyan en los intermedios (lámina 12: cada token se apoya en lo ya escrito). Votar cadenas independientes desperdicia ese apoyo. Es la historia del <think>: la capacidad vino del RL; el prompt sólo la simulaba.




*RL internalizó CoT + exploración + búsqueda (RLVR)*

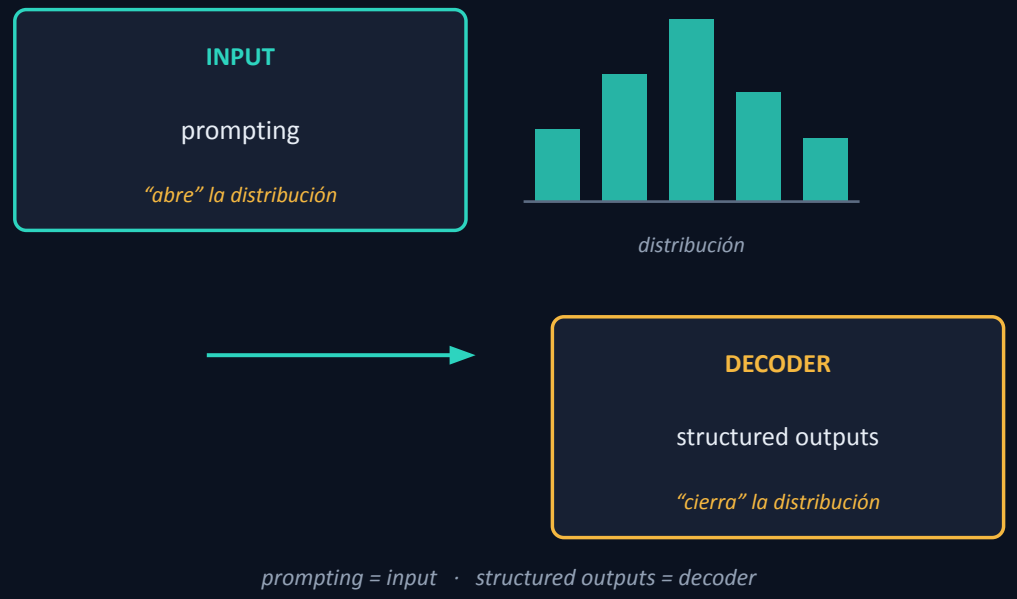
**Guardá "capacidad vs forma".** El razonamiento manual es el ancestro de lo que el RL consolidó adentro. Vuelve en un rato, del otro lado: cuando restrinjamos la salida (structured outputs), la pregunta será si forzar la forma le come capacidad al razonamiento — el mismo eje, invertido.

# El giro: del input al decoder

**Dónde estuvimos.** Todo el prompting —condicionar y expandir— opera sobre el input: qué le mostrás, cuánto lo dejás pensar. Nunca tocamos cómo genera, sólo desde qué genera.

**Adónde vamos.** Structured outputs opera del otro lado: el decoder. Ya no moldeás la condición ni el espacio de razonamiento — intervenís directamente en cómo se elige cada token.

**La simetría que lo conecta.** Hay un puente mecánico: expandir abría la distribución (más caminos para razonar); structured outputs la cierra (prohíbe caminos para garantizar la forma). Operaciones opuestas sobre la misma distribución — una desde el input, la otra desde el decoder. Por eso pueden competir.




**El segundo lugar de intervención.** Dejamos el input y bajamos al decoder. Prompting moldea de dónde sale el token; structured outputs, cuál puede ser. Es el núcleo técnico de AI Engineering — y la mitad del campo que el survey de prompting ni cataloga.

# El problema: el JSON “a mano” se rompe

**El pedido ingenuo.** Le pedís en el prompt: “respondé en JSON (JavaScript Object Notation) con estos campos”. A veces sale perfecto. Otras agrega texto antes (“Claro, acá está:”), rompe una comilla, olvida un campo, o inventa uno de más.

**Por qué falla.** El modelo es un predictor estocástico de tokens: nada garantiza que la secuencia muestreada sea JSON válido. Pedirlo en el prompt es una súplica, no una garantía.

**Por qué es un problema real.** En producción, esa salida alimenta un parser, una base de datos, otra función. Un JSON roto entre mil rompe el pipeline. “Casi siempre válido” no sirve cuando integrás sistemas.



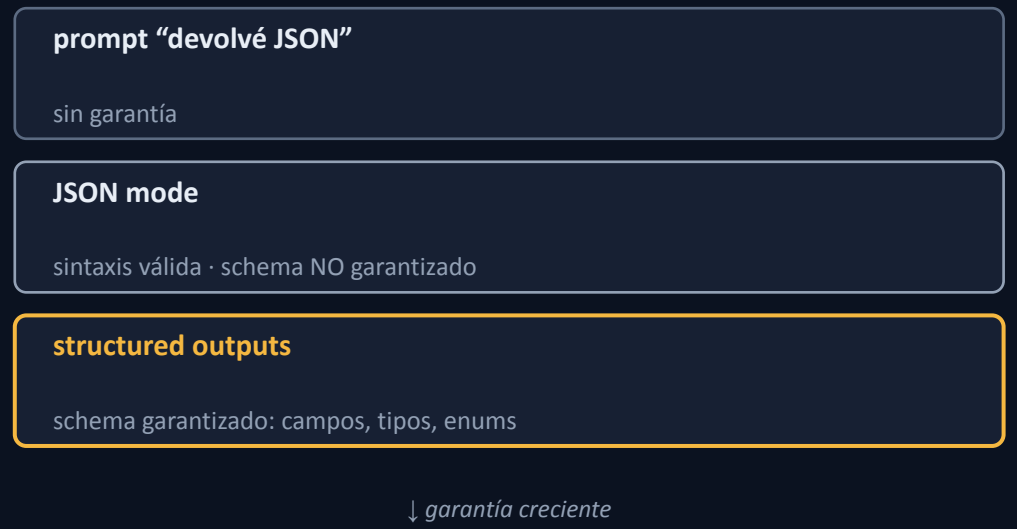

**Lo que necesitamos.** No una súplica que a veces funciona: una garantía. Que sea imposible que la salida no sea JSON válido contra nuestro schema. Eso no se logra pidiendo mejor — se logra cambiando cómo se genera.

# De JSON mode a Structured Outputs

**Primer intento: JSON mode.** La primera solución fue JSON mode: el sistema garantiza que la salida sea JSON sintácticamente válido (parsea), pero no que respete tu schema — puede faltar un campo o sobrar otro.

**El salto: Structured Outputs.** Structured outputs va más allá: garantiza que la salida cumpla exactamente un JSON Schema que vos definís — cada campo, cada tipo, cada restricción. No “es JSON”: es tu JSON.

**La diferencia que importa.** JSON mode elimina el “no parsea”. Structured outputs elimina además el “parsea pero no es lo que pedí”. El segundo es el que te deja borrar la lógica de validación y reintentos.




**La garantía, nombrada.** Structured outputs = la salida cumple tu schema por construcción. La pregunta es cómo se garantiza algo así en un modelo estocástico. La respuesta —el corazón técnico de la clase— viene ahora, y conecta con softmax y con tokenización.

# El mecanismo (1): una distribución sobre el vocabulario

**Volvamos al decoder.** En cada paso, el modelo produce logits sobre todo el vocabulario y un softmax los vuelve una distribución (Clase 2). Probabilidad no nula para cada token del vocabulario (Clase 3). De ahí samplea el próximo.

**La implicancia clave.** El modelo siempre podría elegir cualquier token: una llave, una letra, un espacio, basura. “Salida estructurada” no hace que el modelo “se esfuerce más” ni entienda mejor el pedido.

**Dónde vamos a intervenir.** Si el problema es que puede elegir cualquier token, la solución es prohibirle los tokens que romperían la forma — antes de samplear. No en el prompt: en el paso de decoding.




*prob. no nula para TODO token · el modelo puede elegir cualquiera*

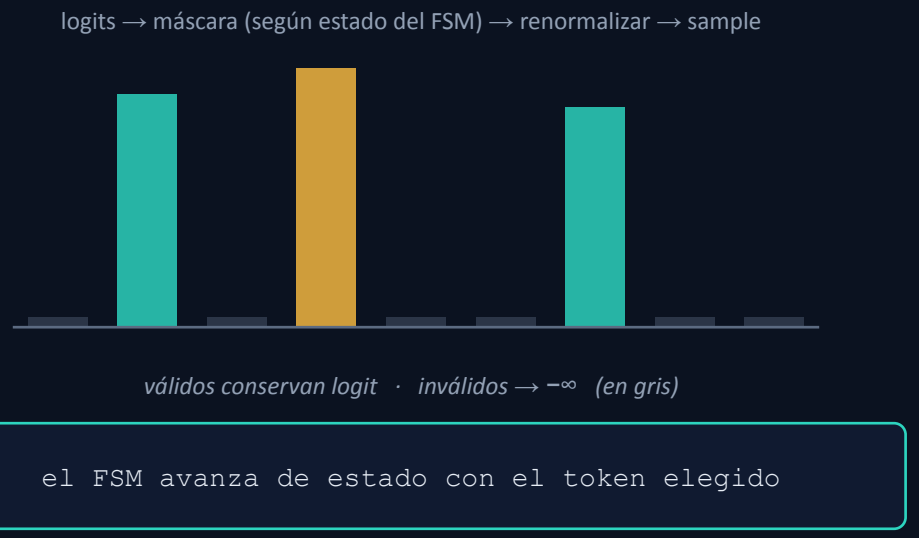
**El punto de intervención.** La forma no se arregla persuadiendo al modelo. Se arregla operando sobre la distribución que ya produce. La próxima lámina es el mecanismo exacto — y es el Aha! central del curso.

# El mecanismo (2): enmascarar logits

**La idea, completa.** Se llama constrained decoding (decodificación restringida). Compilás el schema a una gramática, y la gramática a un finite-state machine (FSM, autómata de estados finitos) que sabe, en cada punto, qué caracteres pueden seguir válidamente.

**La máscara.** En cada paso, a los tokens que romperían la gramática les ponés el logit en $-\infty$ (los enmascarás) antes del softmax. Renormalizás sobre lo que queda, y sampleás. Los tokens inválidos tienen probabilidad exactamente cero.

**Por qué es una garantía y no una súplica.** El modelo, literalmente, no puede emitir un token que rompa la forma: se lo sacaste de la distribución. La garantía vive en el decoder, no en el prompt. El modelo ni se entera.



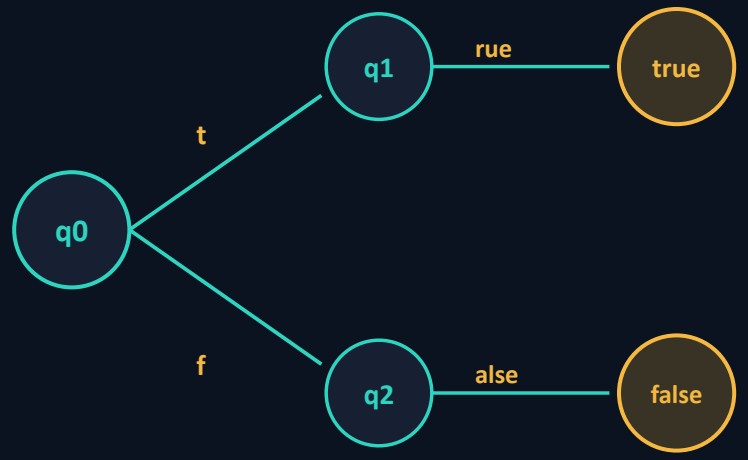

**El Aha! del curso.** El JSON garantizado no es el modelo "esforzándose": es una máscara sobre la distribución en el decoder. Es softmax (Clase 2) + vocabulario (Clase 3) + un autómata que apaga los tokens prohibidos. Persuasión cero; restricción dura.

# El mecanismo, caminado

**Un ejemplo mínimo.** Supongamos que el schema exige un booleano: la salida tiene que ser true o false, nada más. Miremos cómo camina el FSM, token a token.

**Estado inicial.** Sólo dos continuaciones válidas: un token que empiece t (hacia true) o uno que empiece f (hacia false). Todo el resto del vocabulario → máscara. El modelo elige, digamos, t.

**Se avanza y se cierra el abanico.** Desde ahí, la única continuación válida es la que completa true. Cualquier otra cosa → $-\infty$. El FSM llega a un estado final: la salida está completa y válida por construcción. En ningún paso hubo chance de romper la forma.




*todo el resto del vocabulario → enmascarado · sample dentro de lo válido*

**Construir, no postular.** La garantía no es una promesa abstracta: es este abanico que se abre y se cierra en cada token. El modelo sigue eligiendo —hay libertad dentro de lo válido—, pero fuera de la gramática la probabilidad es cero. Escalá esto a un JSON Schema y tenés structured outputs.

# La conexión con tokenización: por qué no es trivial

**El desajuste.** La gramática se define a nivel de caracteres ({, ", un dígito). Pero el modelo no emite caracteres: emite sub-word tokens (Clase 3). Un solo token puede ser " name" —un espacio más varias letras— y cruzar varios estados de la gramática de una.

**Por qué complica.** No alcanza con preguntar "¿este carácter es válido?": tenés que saber, para cada estado del autómata, qué tokens del vocabulario son continuaciones válidas — y un token puede meterte varios pasos adentro del autómata.

**La solución (Outlines, Willard & Louf 2023).** Precomputás un índice: un mapa de cada estado del FSM al conjunto de tokens válidos (σ: Q → conjunto de tokens). Enmascarar es entonces una búsqueda, no recorrer las decenas de miles de tokens del vocabulario.



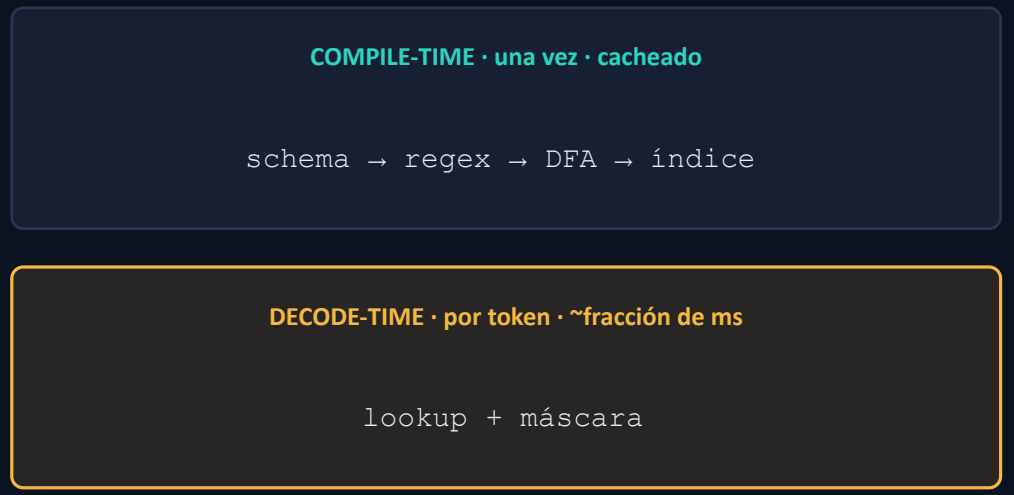

**El mismo problema de tokenización, del lado de la generación.** Que los tokens no sean ni caracteres ni palabras —lo que complicaba el BPE— es lo que hace no-trivial restringir la salida. (Detalle: una context-free grammar (CFG) expresa recursión —JSON anidado— que un FSM plano no; por eso los sistemas serios usan gramáticas, no sólo regex.)

# ¿Es gratis? Compilar una vez, enmascarar por token

**Dos momentos, dos costos.** Compilación (una vez por schema): schema → regex → deterministic finite automaton (DFA) → índice estado-a-tokens. Cuesta un poco y se cachea. Decoding (cada token): un lookup y una máscara.

**El costo por token es despreciable.** Una vez construido el índice, enmascarar agrega del orden de fracciones de milisegundo por token — nada al lado del forward pass del modelo. Restringir sale, en la práctica, casi gratis.

**El resultado contraintuitivo.** Restringir no sólo es barato: puede mejorar la performance en algunas tareas (equipo de Outlines). Sacarle al modelo la opción de irse por las ramas puede ayudarlo a acertar. Guardá este matiz: choca con el debate que viene.




*el costo real es el forward pass, no la máscara*

**El balance de ingeniería.** La garantía de forma es prácticamente gratis en cómputo, y elimina la lógica de validación y reintentos. Ese es el argumento fuerte para usar structured outputs por default en producción — con una salvedad sobre razonamiento que discutimos en un rato.

# El camino práctico: Ollama + Pydantic

**En nuestro stack, gratis.** Desde la v0.5 (dic 2024), Ollama soporta structured outputs: pasás un JSON Schema en el parámetro format. En Python, definís el schema con Pydantic y lo serializás con model_json_schema(). El mismo Qwen3 del notebook.

**Qué pasa por debajo.** Ollama corre los modelos con llama.cpp, que implementa la restricción con gramáticas GBNF (GGML BNF, variante de Backus-Naur Form). Lo que describimos —compilar a gramática y enmascarar— es exactamente lo que ocurre acá abajo.

**El detalle que confirma la teoría.** Se verificó que el schema no se inyecta en el prompt: el modelo no "ve" el schema como texto. La restricción es 100% del decoder. La prueba de que no es persuasión.

```
from ollama import chat
from pydantic import BaseModel

class Pais(BaseModel):        # el schema = una clase
    nombre: str
    capital: str
    idiomas: list[str]

resp = chat(
    model="qwen3",
    messages=[{"role":"user","content":"Contame de Canadá."}],
    format=Pais.model_json_schema(),  # la garantía vive acá
)
pais = Pais.model_validate_json(resp.message.content) # parsea
```

**La teoría, en tres líneas.** format=schema activa la máscara de logits vía GBNF en llama.cpp; el schema nunca toca el prompt. Structured outputs no es un truco de prompt: es una intervención en el decoding. ▸ Notebook (lo corremos con Qwen3).

# Los nombres de los campos son parte del prompt

**El modelo mental equivocado.** Es tentador pensar el schema como un validador externo: el modelo genera, el schema chequea. Falso — y el error tiene consecuencias medibles.

**Por qué.** Las keys del JSON las genera el modelo, como tokens. El nombre de un campo y el orden de los campos están dentro de la secuencia generada: dirigen lo que viene después, igual que cualquier otro token del contexto.

**El caso reportado.** Renombrar un campo de final_choice a answer —misma tarea, mismo prompt, mismos pesos— movió la accuracy de clasificación de forma dramática (Instructor; conviene verificarlo en tu propia tarea). Un nombre de campo movió el resultado porque un nombre de campo es prompt.

**La regla práctica.** Nombrá los campos de forma semántica, y poné los de razonamiento antes que los de respuesta: dejá que el modelo escriba su razonamiento (que es cómputo) antes de comprometer la respuesta. Es “reason-then-structure”.




**El schema no es externo: es prompt.** Sus keys viven en el camino de salida, así que las elegís con el mismo cuidado que cualquier token. Y el orden importa: razonamiento antes que respuesta le devuelve al modelo el lugar para computar que la restricción, mal usada, le saca.

# El debate: ¿restringir degrada el razonamiento?

**La alarma.** Let Me Speak Freely? (Tam et al. 2024) reportó una caída significativa del razonamiento bajo restricciones de formato, y peor cuanto más estricta la restricción. Sonó fuerte: ¿structured outputs rompe el razonamiento?

**La refutación (.txt / dottxt, 2024).** El estudio tenía fallas: la condición "estructurada" era pedir JSON en el prompt (JSON mode), no constrained decoding real; los prompts no estaban emparejados; usaban un LLM como parser. En comparación justa, la generación estructurada iguala o supera a la libre.

**La síntesis (Capacity, Not Format, 2026).** El factor clave no es el formato: es la capacidad sobrante del modelo. Si tiene headroom sobre la tarea, restringir sale gratis. Si la tarea lo empuja cerca de su límite, cumplir el formato compite con el razonamiento por la misma capacidad de generación.




**El puente que reúne las dos palancas.** Expandir y restringir son operaciones sobre la MISMA capacidad de generación. Por eso "razonamiento antes que respuesta" funciona, y por eso capacidad vs forma es el eje de la Clase 6 (el <think>), ahora del lado de la salida. Nunca tragues un solo paper — mirá la réplica y la síntesis.

# Function calling: un schema con forma de función

**Qué es.** Function calling (o tool use) es structured outputs donde el schema es la firma de una función: nombre + argumentos tipados. El modelo, en vez de responder texto, emite un JSON que dice “llamá a get_weather con {ciudad: 'Bariloche'}”.

**Lo que el modelo NO hace.** El modelo no ejecuta nada. Sólo produce —de forma garantizada por la máscara— la intención de llamada, validada contra la firma. Ejecutar la función es trabajo de tu código.

**El loop.** Usuario pregunta → el modelo emite {tool, args} (structured) → tu código corre la herramienta → le devolvés el resultado al modelo → el modelo responde con eso. La API (Application Programming Interface) del modelo estandariza este ida y vuelta.



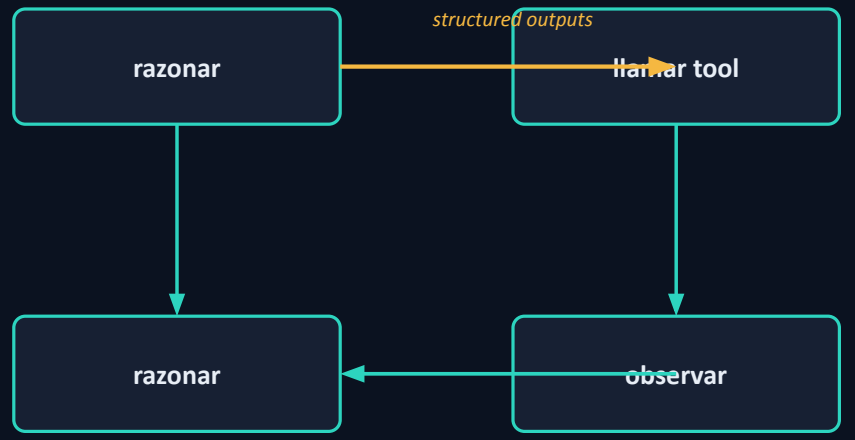

**La misma máscara, otro schema.** Function calling no es una capacidad nueva del modelo: es structured outputs cuyo schema es una firma de función. La confiabilidad de la llamada es la misma garantía de logit-masking que vimos — por eso funciona en producción.

# Por qué esto es el puente a los agentes

**De responder a actuar.** Un modelo que puede pedir herramientas de forma confiable deja de ser un generador de texto y empieza a actuar sobre el mundo: buscar, calcular, consultar una base, llamar una API.

**El esqueleto del agente.** El loop generar → llamar herramienta → observar el resultado → seguir razonando → volver a llamar es, exactamente, el esqueleto de un agente. Junta el razonamiento (primera palanca) con la acción confiable (structured outputs).

**Un cuidado que ya asoma.** Más herramientas no es mejor: pasar de ~20 tools degrada la selección (un Llama 3.1 8B falló un benchmark de tool-use con 46 herramientas y lo pasó con sólo las 19 relevantes). Elegir qué cargar —y no saturar el contexto— es materia de la unidad de agentes.




*la flecha "llamar tool" confiable*

*→ Unidad 5: agentes (selección de tools, memoria, contexto)*

**La precondición de lo agéntico.** Sin structured outputs confiable, no hay tool use confiable; sin tool use confiable, no hay agente que se banque producción. Todo lo agéntico se apoya en la máscara de logits de hoy. ▸ Notebook (el lab: llamadas a herramientas con Ollama + HuggingFace).

# El prompt es código: definí éxito, medí, iterá

**El cambio de mentalidad.** Un prompt no es un hechizo que se ajusta a ojo: es un artefacto que versionás como código y evaluás contra un conjunto de casos. La práctica seria de prompting es eval-driven, no intuición.

**El loop.** Definí qué es "éxito" para tu tarea → armá un set de evaluación → medí el prompt contra él → iterá. Sin métrica, estás optimizando a ciegas. Conecta directo con la Clase 6: evaluar es la mitad del trabajo.

**Por qué importa acá.** Este loop —proponer un prompt, medirlo, mejorarlo— es un proceso de optimización. Y si es optimización… ¿por qué lo haría un humano a mano? Esa pregunta abre la próxima lámina.




*prompt versionado como código*

→ ***¿y si esto lo automatiza un modelo?***

**De arte a ingeniería.** Tratar el prompt como código —con evals y versiones— separa el prompting de producción del de demo. Y una vez que hay una métrica, el paso siguiente es inevitable: automatizar la búsqueda del mejor prompt.

# APE: la instrucción como un programa

**El remate meta.** Automatic Prompt Engineer (APE, Zhou et al. 2022) trata al prompt mismo como el objeto a optimizar. Un LLM propone instrucciones candidatas; se puntúan sobre datos held-out; se selecciona y refina por búsqueda (Monte Carlo con Upper Confidence Bound, UCB).

**El resultado que cierra el arco.** APE encuentra, automáticamente, un prompt de razonamiento mejor que el “let's think step by step” humano. El humano sale del loop: el modelo ingeniería su propio prompt.

**El arco, completo.** Wei (mostrar el razonamiento) → Kojima (una frase gatillo) → APE (el modelo descubre la mejor frase solo). La técnica manual se volvió un problema de optimización, y el optimizador es otro LLM.




**Sale el humano del loop.** APE es el loop de la lámina anterior (definir éxito → medir → iterar) hecho automático, con un LLM como generador y como optimizador. La “ingeniería de prompts” deja de ser artesanal y se vuelve búsqueda.

# El paralelo meta con el alineamiento

**Optimizar contra una señal.** APE optimiza el prompt contra un score. Es el mismo movimiento que optimizar la política contra un reward en RLHF (Reinforcement Learning from Human Feedback) / RLVR (Clase 6): elegir el candidato que maximiza una señal.

**El modelo se opera a sí mismo.** Y hay un eco más profundo. En la Clase 6, el modelo terminaba juzgándose a sí mismo (RLAIF — Reinforcement Learning from AI Feedback, Constitutional AI). Acá termina prompteándose a sí mismo. El humano subió un nivel en los dos casos: de hacer, a definir la señal.

**Por qué es el mismo patrón.** Alineamiento: el modelo genera preferencias y se entrena con ellas. APE: el modelo genera prompts y se selecciona con ellos. La meta-movida —el modelo en el loop de su propia mejora— es la misma.




*el modelo en el loop de su propia mejora · el humano define la señal*

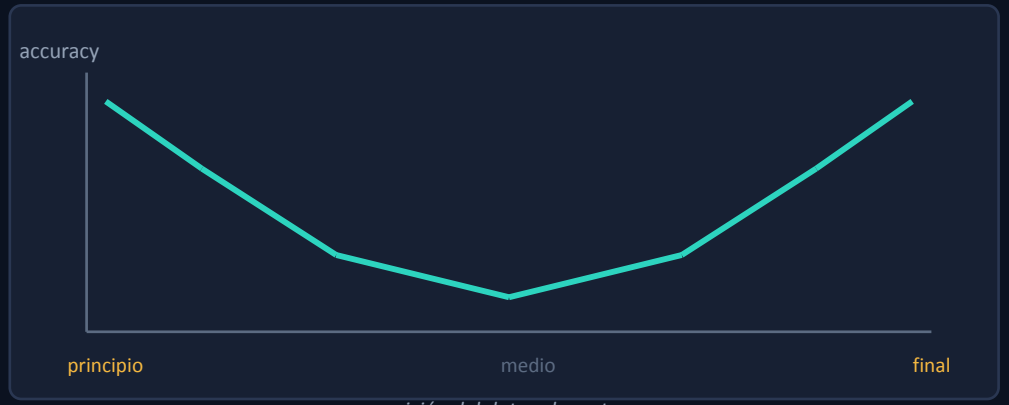
**La rima con la Clase 6.** Modelo que se juzga (RLAIF) ↔ modelo que se promptea (APE). En los dos, el trabajo humano se corre de ejecutar a definir el criterio. Es el mismo patrón meta, en dos capas distintas del stack.

# Dónde ponés la información: no se usa pareja

**Meter un dato no garantiza que se use.** Antes de curar la ventana, un hecho empírico incómodo: dónde ponés la información cambia si el modelo la usa. Lost in the Middle (Liu et al. 2023) lo midió en QA multi-documento y recuperación clave-valor.

**La curva en U.** El rendimiento es más alto cuando la info relevante está al PRINCIPIO o al FINAL, y se degrada en el MEDIO —incluso en modelos de contexto largo—. Es sesgo posicional: primacía y recencia (conecta con la Clase 2: los transformers causales tienden a sesgo de recencia).

**No es sólo la posición: es el largo.** Context Rot (Chroma, 2025) lo generaliza: 18 modelos —incluido Qwen3— y el rendimiento se vuelve menos confiable a medida que crece el input, aún en tareas simples. La ventana grande no es un contenedor neutro.




*Context Rot: la confiabilidad cae con el largo (medido incluso en Qwen3)*

**Curar la ventana tiene una geometría.** No es sólo QUÉ metés: es DÓNDE y CUÁNTO. Poné lo importante arriba o abajo, no sepultado en el medio; menos pasajes bien elegidos le ganan a muchos apilados. Es input directo para RAG (dónde ubicar los chunks recuperados) — próxima clase.

# De prompt a context engineering

**El corrimiento del campo.** El foco se movió de "escribir la oración perfecta" a curar todo lo que entra en la ventana: instrucciones de sistema, documentos recuperados, estado, memoria, herramientas. Anthropic lo formalizó (Effective context engineering for AI agents, 2025): menos encontrar las palabras justas, más "¿qué configuración de contexto hace más probable el comportamiento deseado?".

**Por qué, con raíz en la Clase 2.** El contexto es un recurso finito. La razón está en la atención: el Transformer arma $n^2$ relaciones de a pares para n tokens, así que hay un presupuesto de atención finito que se estira al crecer el contexto. Lost in the Middle y Context Rot son ese presupuesto agotándose. La disciplina: el conjunto más chico de tokens de alta señal que maximiza el resultado.

**El puente hacia adelante.** RAG (Retrieval-Augmented Generation, Clase 8) es context engineering aplicado a qué recuperar; los agentes, a memoria, tools y estado.




*atención $O(n^2)$ = presupuesto finito*

→ RAG (Clase 8) · → agentes (Unidad 5)

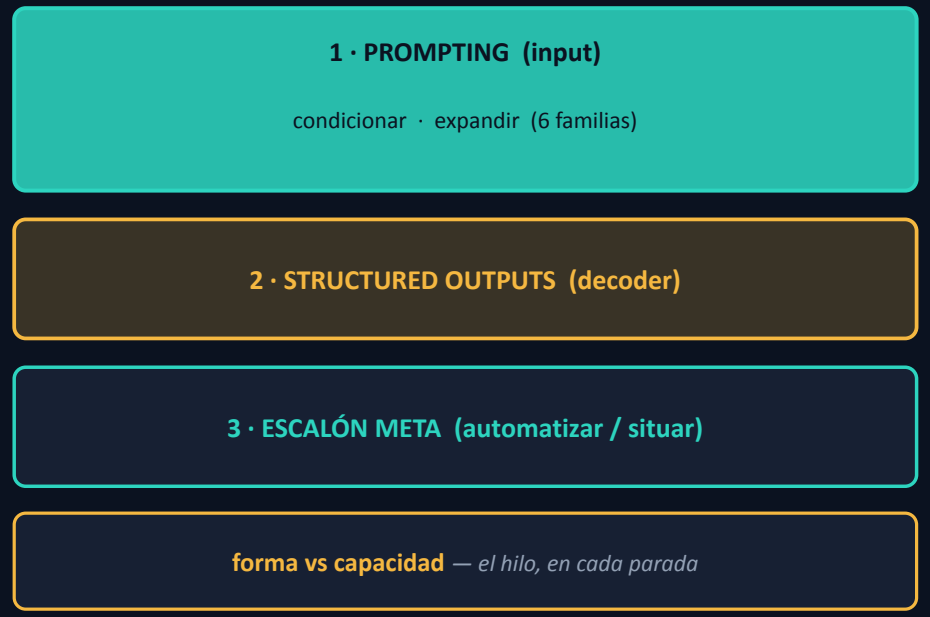
**El prompt, en su lugar.** Prompt engineering no desaparece: se vuelve una capa de algo más grande. Curar la ventana entera —con la atención $O(n^2)$ como límite físico— es el problema real en producción, y es exactamente lo que RAG (recuperar) y los agentes (memoria + tools) van a construir.

# El mapa, recorrido entero

**Los tres lugares.** Recorrimos los tres: prompting (moldear el input) —condicionar y expandir, las seis familias—; structured outputs (intervenir en el decoder) —la máscara de logits—; y el escalón meta —automatizar el prompt (APE) y curar todo el contexto (context engineering)—.

**El hilo mecánico.** Los tres dirigen la misma distribución del próximo token, sin tocar los pesos: desde el input (prompting), desde el decoder (structured outputs), y desde arriba (meta).

**El backbone, atravesándolo todo.** Forma vs capacidad: condicionar y few-shot enseñan la forma; expandir invoca una capacidad que el RL consolidó; structured outputs fuerza la forma y puede competir con la capacidad. La misma tensión de la Clase 6 (el <think>), en cada parada.




**Una sola idea, tres lugares, un backbone.** Dirigir una distribución ya entrenada: desde el input, desde el decoder, y desde arriba. Prompt engineering no es una bolsa de trucos — es esto.

# El mapa ya lo recorriste casi entero

**No te falta un catálogo.** Cinco de las seis familias ya las construimos, cada una con al menos una técnica concreta. La que falta no es un olvido: reaparece en otra unidad.

**La lectura.** Que el curso no enseñe las 58 no es una carencia — es una decisión: una técnica-ejemplar por familia. Y ojo con el alcance: las seis familias son sólo el bucket de prompting — structured outputs no está entre ellas porque es decoder.

**La única que falta.** Self-Criticism —que el modelo critique y revise su salida— no la vimos acá; es el esqueleto de la reflexión en agentes, así que la construimos allá.

| | |
|---|---|
| **Few-Shot** | → el espectro y qué enseñan |
| **Zero-Shot** | → el prompt como condición · zero-shot CoT |
| **Thought Generation** | → Chain-of-Thought |
| **Ensembling** | → self-consistency |
| **Decomposition** | → de la cadena al árbol |
| **Self-Criticism** | → no construida — reaparece en agentes |

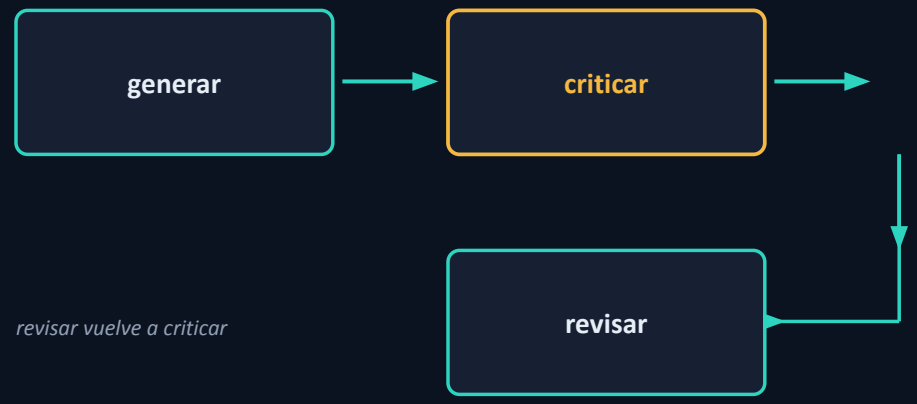
**No te falta un catálogo; te falta, a lo sumo, una familia — y reaparece en agentes.** El miedo a "estar salteándose técnicas" era infundado: ya recorriste el mapa.

# Autocrítica: generar, criticar, revisar

**La idea.** En vez de aceptar la primera salida, el modelo la evalúa y la corrige: produce una respuesta, después la critica, y genera una versión revisada (Self-Refine, Madaan et al. 2023; Chain-of-Verification, Dhuliawala et al. 2023). Es la sexta familia, la que no aparece en el resto de la clase.

**Por qué puede funcionar.** Es una segunda pasada con la primera respuesta a la vista: el modelo gasta más cómputo y se apoya en lo que ya escribió — el mecanismo de "tokens como cómputo", ahora aplicado a revisar en vez de a resolver.

**Cuándo NO funciona.** Si el modelo no puede detectar su propio error, criticarse no lo arregla —y a veces "corrige" una respuesta que estaba bien—. Sube el techo sólo si la capacidad de juzgar ya está: otra vez forma vs capacidad.




*misma máquina que la reflexión (Reflexion) en agentes*

**Es el esqueleto de la reflexión en agentes — por eso es la única familia que conviene sembrar acá.** La construimos en serio en la unidad de agentes.

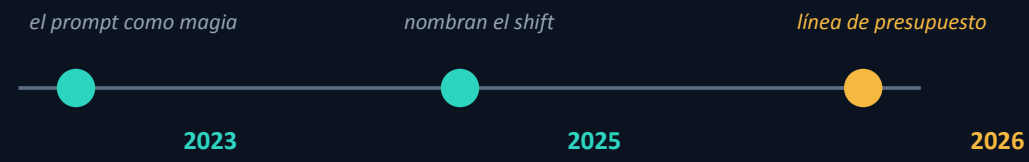
ESTADO DEL ARTE · SEPTIEMBRE 2026

# El campo se reorganizó alrededor del contexto

**El giro que vimos, a escala de industria.** Lo que en clase fue una lámina —de la oración perfecta a curar toda la ventana— el campo entero lo adoptó. Karpathy le puso nombre a mediados de 2025 (context engineering: llenar la ventana con la información justa); Gartner tituló “context engineering is in, prompt engineering is out”; para 2026 dejó de ser debate y es línea de presupuesto.

**Por qué ahora y no en 2024: los agentes.** Las apps single-shot —un prompt, una respuesta— dieron paso a agentes que leen archivos, llaman herramientas, chocan errores y acumulan todo en una ventana creciente. Prompt engineering optimizaba un mensaje; context engineering optimiza todo lo que el modelo ve.

**La nuance, para no comprar el hype.** Buena parte del “prompt engineering murió” es clickbait. No murió: se absorbió en algo más grande. Para el caso simple humano→modelo, promptear bien sigue siendo base.




*el cuello de botella: del modelo al contexto*

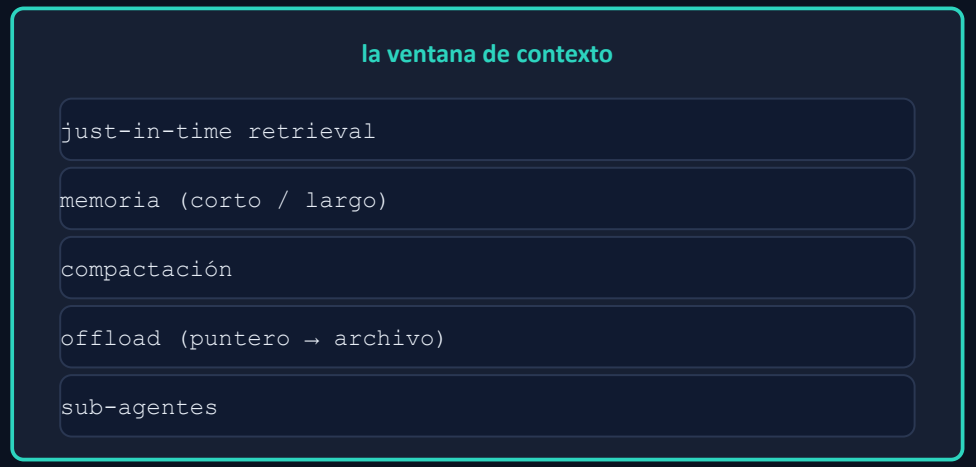
**El centro de gravedad se movió del modelo al contexto.** Prompt engineering no murió — quedó absorbido en algo más grande, que es exactamente el bucket que abrimos con context engineering.

# Las tácticas del contexto (2026)

**Decidir qué ve el modelo en cada llamada.** El oficio tiene técnicas propias: recuperación just-in-time (traer contenido sólo cuando se necesita, con punteros livianos —paths, queries—); memoria de corto plazo (el historial) vs largo plazo (preferencias, convenciones, resúmenes entre sesiones); y compactación (condensar los turnos viejos cuando la ventana se llena).

**No borrar, reubicar.** La tercera vía: mandar los detalles a archivos o a una herramienta de memoria y dejar un puntero en contexto — reversible, y se recupera just-in-time. Más sub-agentes con superficies de herramientas acotadas, para que cada uno tenga contexto enfocado.

**La métrica cambió por el caching.** El objetivo se corrió de “minimizar el tamaño del contexto” a “maximizar el cache hit rate”. Y medido en producción: bajo caching moderno, mantener el historial completo a veces le gana a resumir en costo, latencia y memoria — compactar es respuesta a una restricción, no el default.




*maximizar cache hit rate, no minimizar tokens*

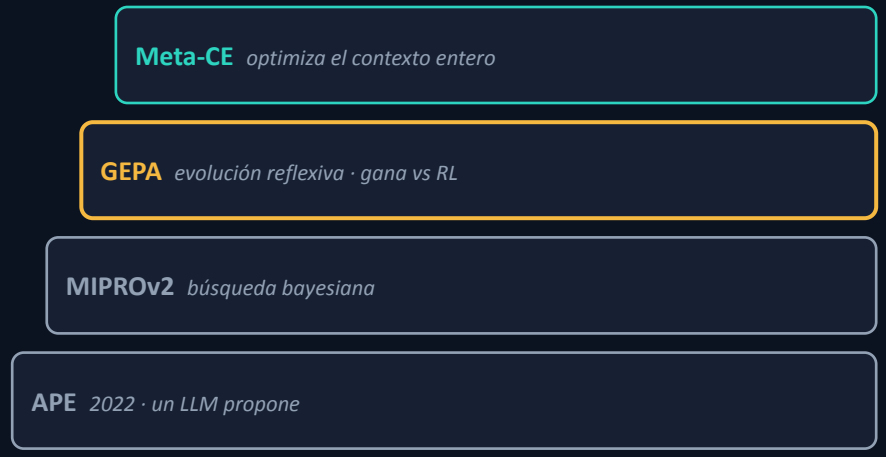
**Curar la ventana es un oficio, no una intuición.** Qué entra, qué se compacta, qué vive afuera con un puntero. Es la unidad de agentes, en preview.

# De APE a GEPA: la máquina le gana a la mano — y al RL

**El remate de la línea meta.** Vimos APE (2022): un LLM que optimiza el prompt. En 2026 el estado del arte es GEPA (Agrawal et al. 2025, oral en ICLR 2026): en vez de colapsar el feedback en un reward escalar, lee las trazas de ejecución, diagnostica en lenguaje natural por qué falló, y evoluciona un frente de Pareto de candidatos.

**El resultado que cierra el arco.** GEPA supera a la búsqueda bayesiana (MIPROv2, +13%) y a métodos de RL (GRPO, +20%) con 35× menos rollouts. La optimización reflexiva de prompts le gana al RL. Y ya escaló al contexto entero: Meta Context Engineering (2026) reporta ~89% en SWE-bench Verified contra ~71% del armado a mano.

**La rima con la clase.** DSPy consolidó "programar, no promptear": el prompt como parámetro aprendible que un optimizador tunea contra una métrica. Es la misma movida meta de la Clase 6 —el modelo en el loop de su propia mejora— ahora ganándole al RL que lo entrenó.




**El prompt (y el contexto) se optimizan solos — mejor que a mano y mejor que con RL.** La "ingeniería de prompts" se volvió, literalmente, un problema de optimización resuelto por otro modelo.

SÍNTESIS · EL HILO

# El hilo con el resto del curso

**Hacia atrás.** Todo se apoyó en lo construido: softmax y máscara causal (Clase 2, que RE2 burla releyendo); tokenización y vocabulario (Clase 3, por qué restringir es no-trivial); los ejes conocimiento/comportamiento (Clase 5); RLVR, forma-vs-capacidad y RLAIF (Clase 6).

**Hacia adelante: donde aterriza el arco de hoy.** El decoder de esta clase es la base de lo agéntico: structured outputs → tool use → MCP (Model Context Protocol, donado a la Linux Foundation en 2025). Los embeddings de la Clase 4 se vuelven la "R" de RAG; el context engineering es la unidad de agentes. Sostiene todo la disciplina de evals y observabilidad.

**El centro de gravedad, y por qué la clase cae ahí.** El campo se movió del modelo al contexto y a los agentes — y esta clase termina en esa bisagra: con el modelo dirigible desde el input, el decoder y el contexto, lo que sigue es darle fuentes (RAG) y acciones (agentes). Es el puente entre entrenar un modelo (Unidades 1–2) y ponerlo a trabajar (Unidades 4–6).

| aporta cada clase | | hacia adelante |
|---|---|---|
| **Clase 2** | softmax · máscara causal | → RAG (Clase 8) |
| **Clase 3** | tokenización · vocab | → agentes (Unidad 5) |
| **Clase 4** | sentence embeddings | → MCP |
| **Clase 5** | conoc. / comportamiento | → evals + observabilidad |
| **Clase 6** | RLVR · RLAIF | |

*del modelo al contexto y a los agentes*

**La bisagra del curso, y del campo.** Con el modelo entrenado, evaluado y dirigible, lo que sigue es fuentes (RAG) y acciones (agentes) — justo hacia donde se movió el estado del arte. Hoy montamos los tres lugares de intervención; las próximas unidades construyen encima.

# Cierre y recursos

**Lo que exhibe el notebook.** Corre la parte demostrable: few-shot y CoT con Qwen3; structured outputs con format + Pydantic (y la verificación de que el schema no se inyecta); y function calling / tool use con Ollama + HuggingFace (el lab).

**Lo que sigue.** La próxima clase, RAG (Retrieval-Augmented Generation): la mitigación más importante de las alucinaciones —traer el hecho de una fuente—, donde los sentence embeddings de la Clase 4 se vuelven búsqueda semántica.


**Fundamentos**
Brown et al. 2022 — few-shot
Min et al. 2022 — qué enseñan las demos

**Razonamiento**
Wei et al. 2022 — Chain-of-Thought
Kojima et al. 2022 — zero-shot CoT
Wang et al. 2022 — self-consistency
Xu et al. 2023 — Re-Reading (RE2)
Yao et al. 2023 — Tree-of-Thought
Prompting Science Report 2 (2025)

**Structured outputs**
OpenAI — Introducing Structured Outputs
Willard & Louf 2023 — Outlines
Tam et al. 2024 — Let Me Speak Freely?
réplica .txt · Capacity, Not Format (2026)

**Contexto**
Liu et al. 2023 — Lost in the Middle
Hong et al. 2025 — Context Rot (Chroma)
Anthropic 2025 — Context engineering
Breunig 2025 + benchmarks (NIAH, RULER, NoLiMa, HELMET)

**Meta y contexto**
Zhou et al. 2022 — APE
Schulhoff et al. 2024 — The Prompt Report

**Estado del arte (2026)**
Agrawal et al. 2025 — GEPA (ICLR 2026)
DSPy — programming, not prompting
Model Context Protocol (Linux Foundation)
Ye et al. 2026 — Meta Context Engineering

**Práctica**
DeepLearning.AI (Fulford & Ng)
docs de prompting de Anthropic
Learn Prompting


**Fuente de varios hilos de hoy** —la escala emergente del razonamiento, la distinción forma-vs-capacidad, el framing de context engineering—: las charlas de Andrej Karpathy (Deep Dive into LLMs, Let's build the GPT Tokenizer). El puente: con el modelo dirigible desde el prompt, lo próximo es darle una fuente. ▸ RAG.